NATIONAL UNIVERSITY OF SINGAPORE

**CS3230: Design and Analysis of Algorithms**

(Semester 1: AY2015/16)

Time Allowed: 120 Minutes

---

**INSTRUCTIONS TO CANDIDATES**

a) This paper consists of **FOUR (4)** questions and **SIX (6)** printed pages including this page. Answer all questions.

b) You have 120 minutes to earn 80 marks. Do not spend too much time on any problem. Read them all through first and attack them in the order that allows you to make the most progress.

c) Show your work, as partial credit will be given. You will be graded on the correctness and efficiency of your answers, and also on your clarity. Be clear and always explain your reasoning.

d) This is a **CLOSED BOOK** exam but you are allowed to bring in **TWO** handwritten (on both sides) A4 crib sheet.

e) You may use an approved calculator.

**Problem 1. True or False, and Justify** (32 Marks)

Circle **T** (true) or **F** (false) for each of the following statements and justify your answer. Your justification is worth more than your true or false answer. You will not receive any mark if the justification is poor or missing, even if you answer true or false correctly.

a) **T F**

Consider the following variant of merge-sort. Instead of recursively sorting two halves of the array and merging them, we divide the array into three equal parts and recursively sort them. Let the outcome of recursively sorting the three parts be $A$, $B$, and $C$ respectively. We merge $A$ and $B$ to get $D$, then merge $D$ and $C$ to obtain $E$, and return $E$ as the outcome. The runtime of this algorithm is still $\Theta(n \lg n)$.

b) **T F**

$T(n) = 4T\left(\frac{n}{2}\right) + \Theta(n \ln n)$ implies $T(n) = \Theta(n \log^2 n)$.

c) **T F**

$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + \Theta(n)$ implies $T(n) = \Theta(n)$.

d) **T F**

It is possible to output the $k$ largest elements in an array in linear time.

e) **T F**

Assume that you would like to sort $n$ integers. The range of the integers to be sorted is in $[1,n^4]$. It is possible to sort the numbers in $O(n)$ time.

f) **T F**

The *amortized cost* of the insert operation in dynamic tables is O(1). Therefore it is appropriate to use for real-time applications where each insert operations need to be done in O(1) *worst case* time.

g) **T F**

The worst case runtime for performing $m$ SEARCH operations for a self-organizing list with $n$ keys using move-to-front is asymptotically as good as the worst case runtime for performing $m$ SEARCH operations for a balanced search tree with $n$ keys because move-to-front is 4-competitive.

h) **T F**

Consider the following algorithm for finding the longest increasing subsequence. We first initialize an array $L[1..n]$ to -1 and $L[1] = 1$. We will use the array $L$ to store the length of the longest increasing subsequence.

```
LIS(A,i)
if L(i) ≠ -1
   return L[i]
max = 1
for j = i-1 downto 1
   x = LIS(A,j)
   if A[i] > A[j] and x + 1 > max
      max = x + 1
L[i] = max
return L[i]
```

The runtime for LIS($A$,$n$) is $O(n)$ as there are only $n$ subproblems to be solved.

**Problem 2. Algorithms in Context** (16 Marks)

The design of efficient algorithms often requires choices. What is efficient in one context may be suboptimal in another. For each of the following, give one context where one algorithm may be preferable to another algorithm together with a reason to support the preference. You should describe contexts that as general as possible. Contexts that are overly restrictive may not be awarded full marks.

> **Example**: When might insertion sort be preferable to merge sort?
> Insertion sort might be preferable to merge sort when space is a limiting resource because unlike merge sort, insertion sort operates in-place.

a) When might merge sort be preferable to radix sort?

b) When might counting sort be preferable to merge sort?

c) When might randomized select be preferable to worst case linear time select?

d) When might memoization be preferable to dynamic programming for solving the same problem?

**Problem 3. Canteen Placement** (24 Marks)

SET-COVER: An instance $(X,F)$ of a set cover problem consists of a finite set $X$ and a family $F$ of subsets of $X$, such that every element of $X$ belongs to at least one subset in $F$, i.e. $X = \bigcup_{S \in F} S$. A subset $C$ covers $X$ if every element of $X$ belongs to at least one subset of $C$, i.e. $X = \bigcup_{S \in C} S$. The set cover problem asks if there is a subset of size $k$ that covers $X$.

VERTEX-COVER: A vertex cover of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that for every $(u, v) \in E$, either $u \in V'$ or $v \in V'$ (or both), i.e. every edge is covered. The vertex cover problem asks if there is a subset $V'$ of size $k$ that covers every edge in $E$.

VERTEX-COVER is known to be NP-complete.

CANTEEN-PLACEMENT: An instance of a canteen placement problem consists of a finite set $S = \{(x_1,y_1), \ldots, (x_n,y_n)\}$ of coordinates corresponding to building locations in an area. The CANTEEN-PLACEMENT problem asks if there is a subset $S' \subseteq S$ of size $k$ such that every element in $S$ is within a Euclidean distance of $d$ from an element in $S'$. This correspond to placing $k$ canteens in $k$ buildings such that there is no building that is located further than a distance $d$ from a canteen.

a) Argue that SET-COVER belongs to NP. (4 marks)

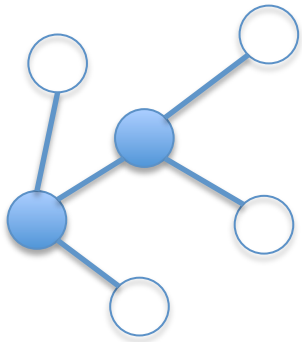
b) Show that SET-COVER is NP-hard, using VERTEX-COVER in the reduction. (12 marks)

c) Show how the CANTEEN-PLACEMENT problem can be formulated as a SET-COVER problem. You may simply describe what the sets in SET-COVER correspond to, without proving that the problems are equivalent. (4 marks)

d) CANTEEN-PLACEMENT can be reduced to SET-COVER in polynomial time. Given that SET-COVER is NP-complete, does that imply that the CANTEEN-PLACEMENT is NP-hard? (4 marks)

**Problem 4. Minimum Vertex Cover for Trees** (8 Marks)

Given a tree $T = (V, E)$, a vertex cover is a subset $U$ of vertices such that for every $(u, v) \in E$, either $u$ or $v$ (or both) is in $U$. The figure below shows a vertex cover for a tree, where the shaded vertices are in $U$.



Argue that the following greedy algorithm will give a vertex cover of the smallest size. At each step, the algorithm maintains a forest $F$, initialized with $T$.

1. Find a leaf $v$ in $F$.
2. Add the vertex $u$ that is adjacent to $v$ (connected to $v$ through edge $(u,v)$) to $U$.
3. Remove all edges and leaf vertices in $F$ that is connected to $u$.
4. Go back to step 1 until all edges have been removed.

**END OF EXAM**